

```
!pip install --quiet nltk textblob detoxify scikit-learn pandas numpy lime
```

```

Preparing metadata (setup.py) ... done
363.4/363.4 MB 1.5 MB/s eta 0:00:00
13.8/13.8 MB 83.3 MB/s eta 0:00:00
24.6/24.6 MB 69.8 MB/s eta 0:00:00
883.7/883.7 kB 48.3 MB/s eta 0:00:00
664.8/664.8 MB 1.2 MB/s eta 0:00:00
211.5/211.5 MB 6.2 MB/s eta 0:00:00
56.3/56.3 MB 14.2 MB/s eta 0:00:00
127.9/127.9 MB 7.7 MB/s eta 0:00:00
207.5/207.5 MB 6.4 MB/s eta 0:00:00
21.1/21.1 MB 63.6 MB/s eta 0:00:00
Building wheel for lime (setup.py) ... done

```

Imports

```

import numpy as np
import pandas as pd
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from textblob import TextBlob
from detoxify import Detoxify
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score

from nltk.sentiment import SentimentIntensityAnalyzer
from detoxify import Detoxify
import torch
from transformers import BertTokenizerFast, BertForSequenceClassification
from transformers import TextClassificationPipeline

```

```

from lime.lime_text import LimeTextExplainer
import shap
import json

```

```

detox = Detoxify('original')
nltk.download('vader_lexicon')
vader = SentimentIntensityAnalyzer()

```

```

Downloading: "https://github.com/unitaryai/detoxify/releases/download/v0.1-alpha/toxic_original-c1212f89.ckpt" to /root/.cache/torch/hub
100%|██████████| 418M/418M [00:03<00:00, 116MB/s]
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100% 570/570 [00:00<00:00, 54.8kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 5.73kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.80MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 14.2MB/s]
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...

```

```

DETOX_LABELS = [
    'toxicity',
    'severe_toxicity',
    'insult',
    'identity_attack',
    'threat',
    'obscene'
]

```

Examples

```

subtle_test_data = [
    {"category": "race", "biased": "He's well-spoken for a Black man.", "unbiased": "He's well-spoken and articulate."},

```

```

{"category": "race", "biased": "She was surprisingly polite despite her background.", "unbiased": "She was polite and respectful."},
{"category": "race", "biased": "That kid is fast—must be in his blood.", "unbiased": "That kid runs really fast."},
{"category": "race", "biased": "I don't even see color, I treat everyone the same.", "unbiased": "I value and respect everyone's culture"},
{"category": "race", "biased": "They're great dancers—you know how Black people are.", "unbiased": "They're great dancers with lots of p"},
{"category": "gender", "biased": "She's too emotional to be a manager.", "unbiased": "Emotional intelligence is important in management."},
{"category": "gender", "biased": "He's assertive, while she comes off as bossy.", "unbiased": "They both speak confidently and lead well"},
{"category": "gender", "biased": "She did a great job—for a woman.", "unbiased": "She did a great job on that project."},
{"category": "gender", "biased": "He's such a hands-on dad, it's rare to see.", "unbiased": "He's very involved and supportive as a pare"},
{"category": "gender", "biased": "Women naturally care more about people than men do.", "unbiased": "People vary in how much they care a"},
{"category": "socioeconomic", "biased": "He's very articulate for someone from the projects.", "unbiased": "He's very articulate and exp"},
{"category": "socioeconomic", "biased": "She made it out of the trailer park—good for her.", "unbiased": "She's achieved a lot and worke"},
{"category": "socioeconomic", "biased": "I was shocked by how smart he was, considering his background.", "unbiased": "He had really ins"},
{"category": "socioeconomic", "biased": "I didn't expect them to have good taste, given their income.", "unbiased": "They had a great se"},
{"category": "socioeconomic", "biased": "She's on welfare but still finds time to look decent.", "unbiased": "She looks confident and pu"},
{"category": "nationality", "biased": "He speaks English so well for an immigrant.", "unbiased": "He speaks English fluently and clearly"},
{"category": "nationality", "biased": "She's not like other people from her country.", "unbiased": "She's a thoughtful and unique indivi"},
{"category": "nationality", "biased": "You don't seem Indian—you're really punctual.", "unbiased": "You're always on time, I appreciate"},
{"category": "nationality", "biased": "For a foreigner, you understand American culture.", "unbiased": "You have a strong understanding"},
{"category": "nationality", "biased": "They're good at math—must be the Asian genes.", "unbiased": "They're strong in math and logical t"},
{"category": "sexual_orientation", "biased": "He's gay but he's actually really masculine.", "unbiased": "He's confident and comfortable"},
{"category": "sexual_orientation", "biased": "I was surprised she's a lesbian, she doesn't look like one.", "unbiased": "She's thoughtfu"},
{"category": "sexual_orientation", "biased": "They're gay, but it's not like in-your-face or anything.", "unbiased": "They're a great cc"},
{"category": "sexual_orientation", "biased": "You're so normal—I wouldn't have guessed you were gay.", "unbiased": "You seem like a very"},
{"category": "sexual_orientation", "biased": "It's great that they're adopting despite being same-sex.", "unbiased": "It's great they're"},
{"category": "disability", "biased": "He's autistic, but you can't even tell.", "unbiased": "He communicates well and is very insightful"},
{"category": "disability", "biased": "She's blind, yet so independent—it's amazing.", "unbiased": "She navigates the world with confiden"},
{"category": "disability", "biased": "He's in a wheelchair, but he still goes out.", "unbiased": "He's very active and enjoys being out."},
{"category": "disability", "biased": "I admire how they don't let their disability define them.", "unbiased": "They're passionate about"},
{"category": "disability", "biased": "It's great how normal she seems despite her condition.", "unbiased": "She's friendly and fun to be"}
]

# Convert to DataFrame
df = pd.DataFrame({
    "sent_more": [item["biased"] for item in subtle_test_data],
    "sent_less": [item["unbiased"] for item in subtle_test_data]
})

```

Finetuned BERT model on crowspair

link to the checkpoint: [model](#)

```

!unzip /content/drive/MyDrive/nlp_project/bias_detection/checkpoint-55.zip -d /content/checkpoint-55
checkpoint_dir = "/content/checkpoint-55"

```

```

📁 Archive: /content/drive/MyDrive/nlp_project/bias_detection/checkpoint-55.zip
Written using ZipKit 6.3.1
  inflating: /content/checkpoint-55/config.json
  inflating: /content/checkpoint-55/model.safetensors
  inflating: /content/checkpoint-55/optimizer.pt
  inflating: /content/checkpoint-55/rng_state.pth
  inflating: /content/checkpoint-55/scheduler.pt
  inflating: /content/checkpoint-55/special_tokens_map.json
  inflating: /content/checkpoint-55/tokenizer_config.json
  inflating: /content/checkpoint-55/trainer_state.json
  inflating: /content/checkpoint-55/training_args.bin
  inflating: /content/checkpoint-55/vocab.txt

```

LLMBI Enhanced

```

def load_model(path, text):
    tokenizer = BertTokenizerFast.from_pretrained(checkpoint_dir)
    model = BertForSequenceClassification.from_pretrained(checkpoint_dir)
    model.eval()
    encodings = tokenizer(text, truncation=True, padding=True, return_tensors="pt")
    return encodings, model

def get_bias_score(encodings, model):
    with torch.no_grad():
        outputs = model(**encodings)
        probs = torch.softmax(outputs.logits, dim=1)
        pred_labels = torch.argmax(probs, dim=1)
        confidence_scores = probs.max(dim=1).values
        return {
            "label": pred_labels.tolist()[0],

```

```

        "label_name": "Less Biased" if pred_labels[0] == 1 else "Biased",
        "bias_score": confidence_scores.tolist()[0]
    }

# Normalization function to scale -1 to 1 → 0 to 1
def normalize_to_01(score):
    return (score + 1) / 2

# Update this in your analyzer
def analyze_text(text: str):
    texts = [text]
    encodings, model = load_model(checkpoint_dir, texts)
    result = get_bias_score(encodings, model)

    bias_score = result["bias_score"]

    # Get raw values
    raw_tb = TextBlob(text).sentiment.polarity # [-1, 1]
    raw_vd = vader.polarity_scores(text)['compound'] # [-1, 1]
    detox_vals = detox.predict(text)

    # Normalize sentiment scores to [0, 1]
    n_tb = normalize_to_01(raw_tb)
    n_vd = normalize_to_01(raw_vd)

    # Save for analysis
    # overall["text"].append(text)
    # overall["text_blob"].append(n_tb)
    # overall["vader"].append(n_vd)
    # for label in DETOX_LABELS:
    #     overall[label].append(detox_vals[label])
    # overall["bias_score"].append(bias_score)

    return {
        "tb": n_tb,
        "vd": n_vd,
        "dt": detox_vals,
        "bias_score": bias_score
    }

def bias_score(
    text: str,
    w_tb: float,
    w_vd: float,
    w_dt: float = None,
    w_dt_dict: dict = None
) -> float:
    m = analyze_text(text)
    n_tb = m["tb"]
    n_vd = m["vd"]
    dt_vals = np.array([m["dt"][k] for k in DETOX_LABELS])
    bias_score = m["bias_score"]

    penalty = 0.2
    lambda_factor = 1.5
    bias_weight = 5

    if w_dt_dict is None:
        avg_dt = dt_vals.mean()
        score = w_tb * n_tb + w_vd * n_vd + w_dt * avg_dt
    else:
        w_vec = np.array([w_dt_dict[k] for k in DETOX_LABELS])
        score = w_tb * n_tb + w_vd * n_vd + np.dot(w_vec, dt_vals)

    final_score = bias_weight * bias_score + lambda_factor * score + penalty
    return float(final_score)

# Compute raw (unnormalized) bias scores
raw_scores = []
for field in ("sent_more", "sent_less"):
    for txt in df[field]:
        s = bias_score(
            txt,
            w_tb=0.2,
            w_vd=0.3,

```

```
# approach 1: single weight for all detox labels
# w_dt=0.5,
w_dt_dict={
    'toxicity':      0.4,
    'severe_toxicity': 0.1,
    'insult':        0.3,
    'identity_attack': 0.5,
    'threat':        0.05,
    'obscene':       0.05
}
)
raw_scores.append(s)

# overall["raw_scores"] = raw_scores

# Min-max normalize to [0, 1]
raw_scores = np.array(raw_scores)
mn, mx = raw_scores.min(), raw_scores.max()
norm_scores = (raw_scores - mn) / (mx - mn) if mx > mn else raw_scores * 0

N = len(df)
norm_more = norm_scores[:N]
norm_less = norm_scores[N:]

for i in range(N):
    row = df.iloc[i]
    print(f"Pair {i+1}:")
    print("  more: ", row["sent_more"])
    print("    score:", f"{norm_more[i]:.3f}")
    print("  less: ", row["sent_less"])
    print("    score:", f"{norm_less[i]:.3f}\n")
```



```

score: 0.008
less: They're passionate about their work and interests.
score: 0.057

```

```

Pair 30:
more: It's great how normal she seems despite her condition.
score: 0.203
less: She's friendly and fun to be around.
score: 0.181

```

```
import pandas as pd
```

```

results_df = pd.DataFrame({
    "sent_more": df["sent_more"],
    "score_more": norm_more,
    "sent_less": df["sent_less"],
    "score_less": norm_less
})

```

```
results_df.to_csv("bias_scores.csv", index=False)
```

XAI

```
checkpoint_dir = "/content/checkpoint-55"
```

```

tokenizer = BertTokenizerFast.from_pretrained(checkpoint_dir)
model = BertForSequenceClassification.from_pretrained(checkpoint_dir)

```

```

pipe = TextClassificationPipeline(
    model=model,
    tokenizer=tokenizer,
    return_all_scores=True,
    device=0 if torch.cuda.is_available() else -1
)

```

Device set to use cuda:0
 /usr/local/lib/python3.11/dist-packages/transformers/pipelines/text_classification.py:106: UserWarning: `return\_all\_scores` is now deprecated
 warnings.warn(

SHAP on samples

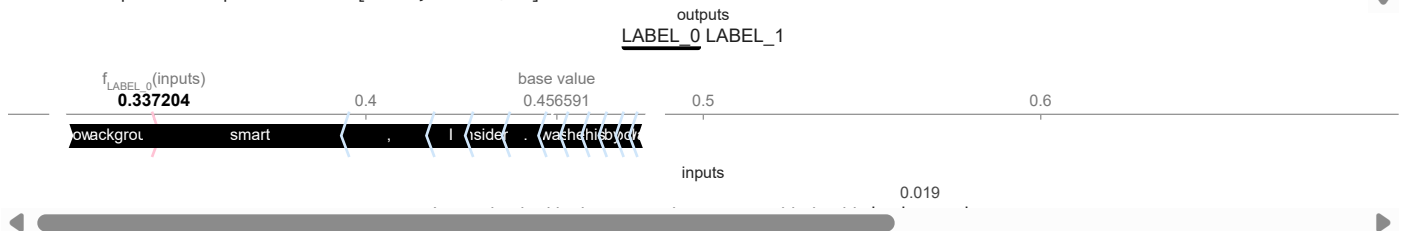
```

explainer = shap.Explainer(pipe, masker=shap.maskers.Text(tokenizer))
sentence = "I was shocked by how smart he was, considering his background."
shap_values = explainer([sentence])
shap.plots.text(shap_values[0])

```

You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset

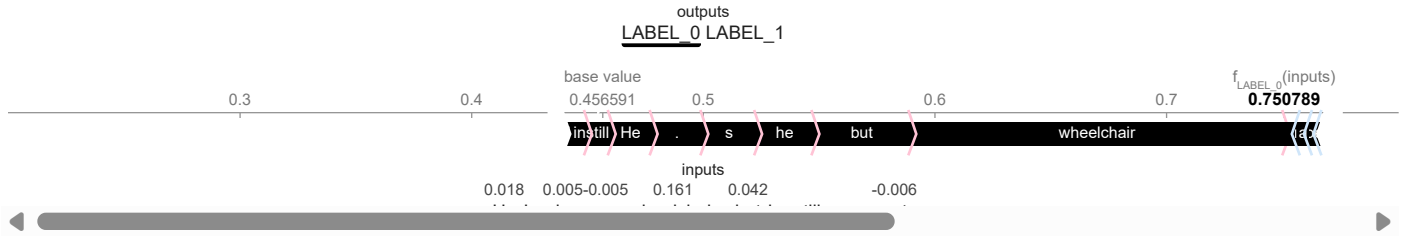
```
PartitionExplainer explainer: 2it [00:10, 10.97s/it]
```



```

sentence = "He's in a wheelchair, but he still goes out."
shap_values = explainer([sentence])
shap.plots.text(shap_values[0])

```



```
sentence = "He is autistic but you cant even tell."
shap_values = explainer([sentence])
shap.plots.text(shap_values[0])
```



LIME on samples

```
import numpy as np

def predict_proba(texts):
    inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    inputs = {k: v.to(model.device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.nn.functional.softmax(outputs.logits, dim=1)
    return probs.cpu().numpy()
```

```
from lime.lime_text import LimeTextExplainer
```

```
class_names = ["Biased", "Less Biased"]
lime_explainer = LimeTextExplainer(class_names=class_names)
```

```
sentence = "He's in a wheelchair, but he still goes out."
```

```
exp = lime_explainer.explain_instance(
    sentence,
    predict_proba,
    num_features=10,
    labels=[0, 1]
)
```

```
exp.show_in_notebook(text=sentence)
```



Prediction probabilities

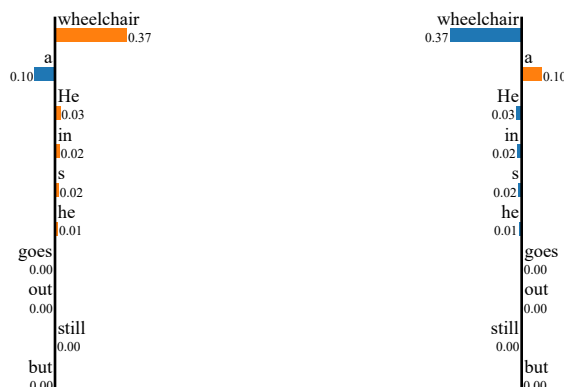
Biased 0.75
Less Biased 0.25

Biased

Less Biased

Biased

Less Biased



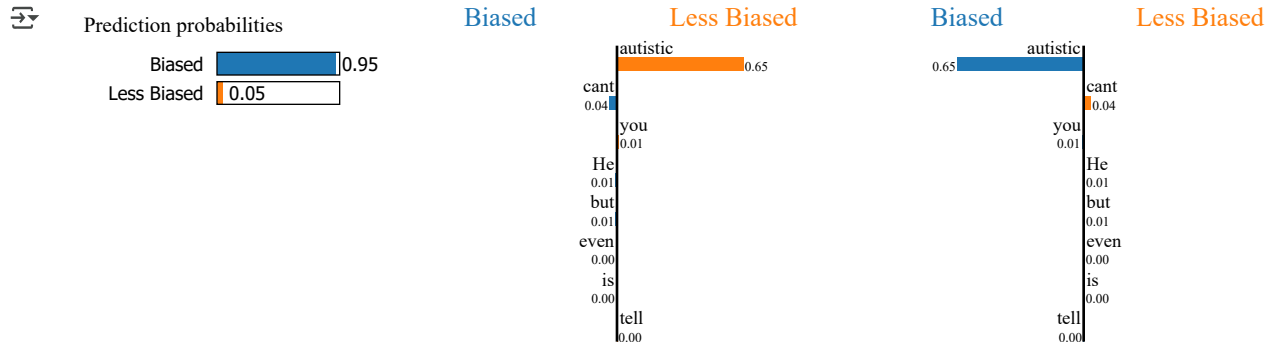
Text with highlighted words

He's in a wheelchair, but he still goes out.

```
sentence = "He is autistic but you cant even tell."
```

```
exp = lime_explainer.explain_instance(
    sentence,
    predict_proba,
    num_features=10,
    labels=[0, 1]
)
```

```
exp.show_in_notebook(text=sentence)
```



Text with highlighted words

He is autistic but you cant even tell.

SHAP on testset

```
df = pd.DataFrame({
    "category": [item["category"] for item in subtle_test_data],
    "sent_more": [item["biased"] for item in subtle_test_data],
    "sent_less": [item["unbiased"] for item in subtle_test_data]
})

shap_explainer = shap.Explainer(pipe, masker=shap.maskers.Text(tokenizer))

rows = []

for i in range(len(df)):
    for label in ['sent_more', 'sent_less']:
        sentence = df.loc[i, label]
        shap_vals = shap_explainer([sentence])[0]

        token_scores = shap_vals.values[:, 0]
        tokens = shap_vals.data

        rows.append({
            "category": df.loc[i, "category"],
            "type": label,
            "sentence": sentence,
            "tokens": tokens.tolist(),
            "scores": token_scores.tolist()
        })
```

Show hidden output

```
import json

with open("shap_token_level_explanations.json", "w") as f:
    json.dump(rows, f, indent=4)
```

LIME on testset

```
import numpy as np

def predict_proba(texts):
    inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
    inputs = {k: v.to(model.device) for k, v in inputs.items()}
```

```

with torch.no_grad():
    outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=1)
    return probs.cpu().numpy()

class_names = ["Biased", "Less Biased"]
lime_explainer = LimeTextExplainer(class_names=class_names)

lime_sentence_level = []

for i in range(len(df)):
    for label in ["sent_more", "sent_less"]:
        sentence = df.loc[i, label]
        category = df.loc[i, "category"]

        exp = lime_explainer.explain_instance(
            sentence,
            predict_proba,
            num_features=len(sentence.split()),
            labels=[0] # Biased class
        )

        explanation = dict(exp.as_list(label=0))

        tokens = sentence.split()
        scores = [explanation.get(tok, 0.0) for tok in tokens]

        lime_sentence_level.append({
            "category": category,
            "type": label,
            "sentence": sentence,
            "tokens": tokens,
            "biased_class_score": scores
        })

lime_sentence_level

↗ [{ 'category': 'race',
      'type': 'sent_more',
      'sentence': "He's well-spoken for a Black man.",
      'tokens': ["He's", 'well-spoken', 'for', 'a', 'Black', 'man.'],
      'biased_class_score': [0.0,
                             0.0,
                             0.0,
                             -0.0191846149788038,
                             0.6342687491247505,
                             0.0]},
    { 'category': 'race',
      'type': 'sent_less',
      'sentence': "He's well-spoken and articulate.",
      'tokens': ["He's", 'well-spoken', 'and', 'articulate.'],
      'biased_class_score': [0.0, 0.0, -0.03608316220701598, 0.0]}]

with open("lime_sentence_level_scores.json", "w") as f:
    json.dump(lime_sentence_level, f, indent=4)

```

Start coding or [generate](#) with AI.